Homework 6: Turing Machine Variants

Will Griffin

March 27, 2026

1. Doubly infinite tapes

A Turing machine with a doubly infinite tape is like a TM but with a tape that extends infinitely in both directions. Initially, the head is at the first symbol of the input string, but there are infinitely many blanks to the right.

Show how, given a TM with a doubly infinite tape, to construct an equivalent standard, single-tape TM. An **implementation description** in the style of Proof 3.13 is fine, and it's also fine to use any results proved in the book or in class.

Given a doubly infinite taped TM D, an equivalent standard TM M can be constructed with the instruction set

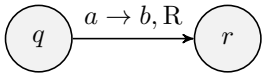
$M =$ "On input w

1. Shift every symbol to the right by one space. Then, move the head to the left end and write some symbol c_{R} where c_{R} (and future c_{L}) $\notin \Gamma_D$ to mark the end of the tape.
2. Then, shift symbols of the input string w such that there is a $\sqcup$ between every symbol of w.
3. Move the head to the first symbol of w.
4. M will remember which side of the tape it is on in reference to D. This is done by switching between c_{R} and c_{L} as the first symbol.
5. Simulate D, where left and right moves of the head of D follow the following rules:
    a. While the first symbol is c_{R},
        i. If the next transition is a move to the right, move the head right two spaces.
        ii. If the next transition is a move to the left and the head is not on w_1, move the head left two spaces.
        iii. If the next transition is a move to the left and the head is on w_1, move the head left once, write c_{L}, then move the head to the right twice.
    b. While the first symbol is c_{L},
        i. If the next transition is a move to the left, move the head right two spaces.
        ii. If the next transition is a move to the right, move the head to the left twice. If the head then reads c_{L}, write c_{R} and then move the head to the right once.

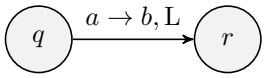
2. Two-stack PDAs

Show that any Turing machine M can be converted into an equivalent 2PDA P. Use **formal descriptions** of both M and P. Be sure to include in your construction the following:

- For each state q of M, you should create a state q in P.
- If s is the start state of M, what should you do?
- If q_{accept} is the accept state of M, what should you do?
- If q_{reject} is the reject state of M, what should you do?
- For each transition that looks like this, what should you do?

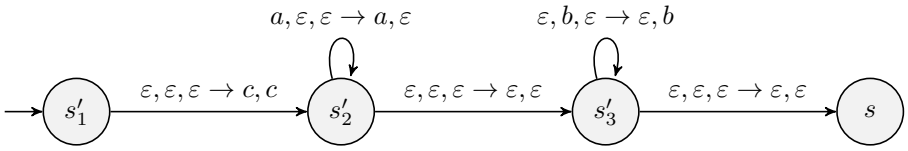


- For each transition that looks like this, what should you do?



A 2PDA P that is equivalent to a standard Turing machine M can be constructed by the following steps:

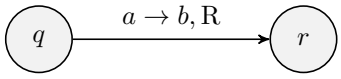
- For each state q in M, create state q in P.
- For the start state s in M, create new states s'_1, s'_2, s'_3 in P with the following transitions:



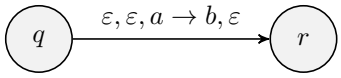
  where $c \notin \Gamma$ and is an end marker, $a, b \in \Sigma$, where transitions including a and b are created for all $a, b \in \Sigma$.

  Essentially, this means that upon reaching s, the first stack will contain c, a terminal character not in the tape alphabet, and the second stack will contain w^{R} where w is the input string and w_1 is the top item in the stack.

- For the accept state q_{accept} in M, make q_{accept} an accept state in P.
- For the reject state q_{reject} in M, ensure state q_{reject} exists in P.
- For each right shift transition in M,

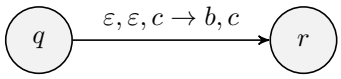


  where q, r are states in M, create a transition in P such that



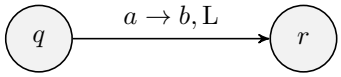
  where q, r are states in P and $a, b \in \Gamma$.

  When $a = \text{␣}$, also add the transition

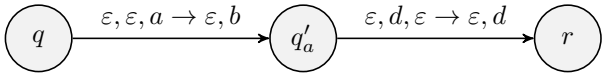


where $c \notin \Gamma$ is a stack end marker.

- For each left shift transition in M,



where q, r are states in M, create transitions in P such that

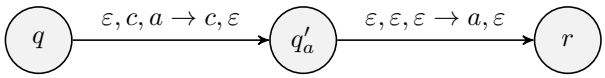


where q, r are states in P and $a, b, d \in \Gamma$.

When $a = \sqcup$, also add the transitions

for all $b \in \Gamma$.

To ensure the "tape does not go past the left end" in P, create states q'_a (for all $a \in \Sigma$) and transitions



where q, r are states in P, $c \notin \Gamma$ and is an end marker, and $a \in \Sigma$. These transitions are created for all $a \in \Sigma$.

3. Brain Fun

Give an **implementation description** of a multitape Turing machine that can interpret any $\mathcal{P}''$ program. The input would be a string $P\#w$ where P is a $\mathcal{P}''$ program and w is an input string, and it should accept iff P accepts w.

A multitape Turing machine M that can interpret $\mathcal{P}''$ programs can be defined by such:

M = "On input u, where $u = P\#w$

1. Copy the w part of the input string u to the second tape. This can be done by first moving head 1 right to the #, then right once more. Then, take whatever symbol $a \in w$ and write it to the second tape, replacing it on the first tape with a $\sqcup$, then moving right head 1 and 2 to the next symbol. Repeat until all symbols of w are read and copied to the second tape.
2. Move head 1 left until it reaches the #, and replace it with a $\sqcup$. Then, move head 1 left until it reaches the left end of the tape. Move head 2 left until it reaches the left end of the tape.
3. Read head 1's symbol, and perform a different action depending on the symbol read:
   - If < is read, move head 2 to the left by one cell if it is not at the left end.
   - If > is read, move head 2 to the right by one cell.

- If `+` is read, read the symbol under head 2 and write it incremented by one (that is, a_0 becomes a_1, a_1 becomes a_2, and so on; a_{n-1} becomes a_0).
- If `-` is read, read the symbol under head 2 and write it decremented by one (that is, a_{n-1} becomes a_{n-2}, a_{n-2} becomes a_{n-3}, and so on; a_0 becomes a_{n-1}).
- If `[` is read, read the symbol under head 2. If the symbol is a_0, move head 1 to the right until it reaches its matching `]`. Otherwise, if the symbol is anything but a_0, continue.
- If `]` is read, move head 1 left until it reaches its corresponding `[`. Then, move head 1 one more cell to the left, so that the condition check can happen at the start of the loop.
- If ␣ is read, read the symbol under head 2. If the symbol is a_0, *reject*, otherwise, *accept*.

4. Move head 1 to the right once and go back to step 3.